

What argument is passed to the recursive function call? Lists made from the left and right halves of the original list to sort.

How does this argument become closer to the base case? The lists passed to the recursive call are half the size of the original list, so they eventually become a list of zero or one item.

The following `mergeSort()` function in the *mergeSort.py* Python program sorts the values in the `items` list into ascending order:

```
import math

def mergeSort(items):
    print('.....mergeSort() called on:', items)

    # BASE CASE - Zero or one item is naturally sorted:
    if len(items) == 0 or len(items) == 1:
        return items ❶

    # RECURSIVE CASE - Pass the left and right halves to mergeSort():
    # Round down if items doesn't divide in half evenly:
    iMiddle = math.floor(len(items) / 2) ❷

    print('.....Split into:', items[:iMiddle], 'and', items[iMiddle:])

    left = mergeSort(items[:iMiddle]) ❸
    right = mergeSort(items[iMiddle:])

    # BASE CASE - Returned merged, sorted data:
    # At this point, left should be sorted and right should be
    # sorted. We can merge them into a single sorted list.
    sortedResult = []
    iLeft = 0
    iRight = 0
    while (len(sortedResult) < len(items)):
        # Append the smaller value to sortedResult.
        if left[iLeft] < right[iRight]: ❹
            sortedResult.append(left[iLeft])
            iLeft += 1
        else:
            sortedResult.append(right[iRight])
            iRight += 1

    # If one of the pointers has reached the end of its list,
```